arXiv:2309.00754v1 [cs.LG] 1 Sep 2023

EFFICIENT RLHF: REDUCING THE MEMORY USAGE OF PPO

Michael Santacroce, Yadong Lu, Han Yu, Yuanzhi Li, Yelong Shen
Microsoft
{misantac,yadonglu,hanyu,yuanzhili,yelong.shen}@microsoft.com

ABSTRACT

Reinforcement Learning with Human Feedback (RLHF) has revolutionized language modeling by aligning models with human preferences. However, the RL stage, Proximal Policy Optimization (PPO), requires over 3x the memory of Supervised Fine-Tuning (SFT), making it infeasible to use for most practitioners. To address this issue, we present a comprehensive analysis the memory usage, performance, and training time of memory-savings techniques for PPO. We introduce Hydra-RLHF by first integrating the SFT and Reward models and then dynamically turning LoRA "off" during training. Our experiments show: 1. Using LoRA during PPO reduces its memory usage to be smaller than SFT while improving alignment across four public benchmarks, and 2. Hydra-PPO reduces the latency per sample of LoRA-PPO by up to 65% while maintaining its performance. Our results demonstrate that Hydra-PPO is a simple and promising solution for enabling more widespread usage of RLHF.

1 Introduction

Since ChatGPT, GPT-4, and Llama-2 family models entered the public sphere, they have impressed users with their ability to be helpful assistants for a surprising number of tasks [1, 2, 3, 4, 5]. One key to their success, along with many other foundation models [6], is model alignment through RLHF. Training a massive language model results in a network with a large amount of knowledge, however, it is not trained to *discriminate within* that knowledge, which could cause undesired behaviour and possibly lead to societal harm [7]. Alignment aims to solve this issue by adjusting the model's behaviour and has become an integral part for creating safe and controllable foundation models [8, 9].

While RLHF improves model alignment it is limited in usage, being both highly complex and demanding a massive amount of memory when loading and training multiple models during PPO [10, 11]. Because the use of RLHF is in its infancy, there is a strong need to evaluate its variations in terms of speed and performance.

To address this need, we delve into the training process and model architectures of standard RLHF-PPO. Through this investigation, we identify substantial opportunities for memory/computation cost reduction through the implementation of model-sharing between Reference/Reward Models and Actor/Critic Models.

Given these findings, we propose Hydra-PPO to reduce the number of trained and static models in memory during PPO. We perform run-time and performance comparisons to show these memory savings can then be utilized to increase the training batch size, reducing the per-sample latency of PPO by up to 65%.

Preprint.

Method	Batch Size	GPU Memory (GB)			Latency per Sample (seconds)		
		Model	Activation	Total	Inference	Update	Total
PPO	1	111.8*	101.3*	220*	-	-	-
LoRA-PPO	1	53.2	12.5	68.0	17.23	1.52	18.75
J-Hydra-PPO	4	14.3	51.4	67.9	4.63	0.38	**5.01**
Hydra-PPO	4	15.9	52.8	71.1	4.88	1.59	**6.47**

Table 1: Comparison of Memory Usage and Run-Time between methods for Llama 7b on StackExchange per A100 80GB GPU. See Appendix B for details. *For Full Fine-Tuning PPO, memory usage is a scaled-up estimate.

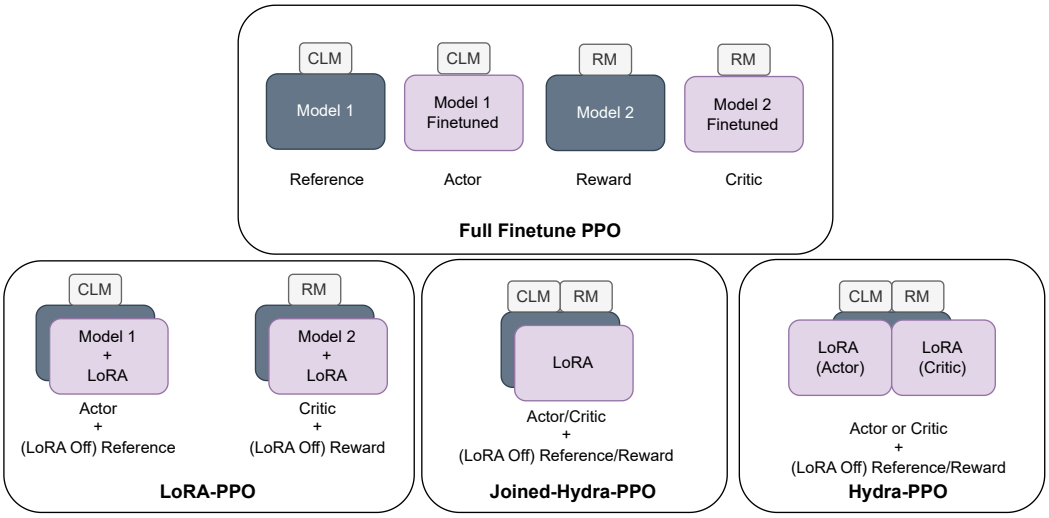

Figure 1: Models used in PPO methods. CLM indicates a Causal Language Modeling head, RM indicates a Reward Modeling head. Light purple weights are trained and dark blue weights are frozen.

2 RLHF

In this section, we first introduce the standard RLHF method [12, 10, 11, 13].

Stage 1: Supervised Fine-Tuning (SFT) an input LLM is trained using the standard causal language model training objective $\mathcal{L}_{\text{xent}}$ on a set of data $\mathcal{D}$, yielding language model π_{SFT}. We call this FFT-SFT when all parameters are trained, or LoRA-SFT when using LoRA [14].

Stage 2: Reward Model (RM) Training the head of a LLM is replaced with a scalar output. This model $r_\phi(x, y)$ is trained to predict human preference given a dataset of preference pairs with prompt x and completion y. After training, the reward function is often normalized such that $\mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_{\text{SFT}}(y|x)}[r_\phi(x)] = 0$ to improve PPO stability. The reward model is trained with loss $\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log(\,\sigma(r_\phi(x,\, y_w) - r_\phi(x,\, y_l)\,)]$, where y_w is the "winning" answer as compared to y_l for prompt x, according to the target alignment source.

Stage 3: PPO π_{SFT} and $r_\phi(x, y)$ are used to initialize and subsequently train an actor and critic with PPO [11, 10]. During training, there are at minimum[1] four models used:

- **Reference:** π_{ref}, a frozen copy of π_{SFT}, used to prevent reward divergence.
- **Actor:** called π_θ, the trained generative model or policy, initialized as a copy of π_{SFT}.
- **Reward:** a frozen copy of $r_\phi(x, y)$, used to calculate the reward of outputs from the Actor.
- **Critic or Value Function:** $V(x, y)$, a copy of $r_\phi(x, y)$ trained to estimate sequence returns.

[1]Other models may be added [10]; we stick to the most common and simplest setup in our paper.

Using output probability ratio $r(\theta) = \frac{\pi_\theta(y \mid x)}{\pi_{\text{old}}(y \mid x)}$, PPO optimizes the surrogate objective $\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r(\theta)\hat{A}, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)\hat{A}]$. Generalized advantage estimation uses $V(x, y)$ to construct advantage estimates $\hat{A}$ from the reward [15, 16]. $V(x, y)$ is trained with squared-error loss on the returns. We use LoRA [14] on all linear layers of π_θ and $V(x, y)$, which we call LoRA-PPO. We do not perform experiments with Full Fine-Tuning PPO due to its extreme cost.

3 Hydra-RLHF

We introduce Hydra-RLHF as a set of modifications to RLHF. We define a decoder-based model π^{hydra} with two linear heads: 1) a head serves as the causal head, predicting the subsequent token for a sequence, and 2) another head serves as the reward model head, providing the immediate reward associated with the same input. Multi-headed models are well-explored both in general [17, 18] and with respect to reinforcement learning [16, 19, 20].

Stage 1: Hydra-SFT Using a similar dataset to standard RM training, π^{hydra} is trained by optimizing $\mathcal{L}_{\pi^{\text{hydra}}}(x, y_w, y_l) = \mathcal{L}_{\text{xent}}(x, y_w) + \gamma \mathcal{L}_\theta(x, y_w, y_l)$, where γ is a weighting multiplier. In practice, we find $\gamma = 0.1$ generally works well. We call this Hydra-FFT when training all parameters.

There are additional requirements for π^{hydra} compared to regular RM or SFT fine-tuning. $\mathcal{L}_{\pi^{\text{hydra}}}(x, y_w, y_l)$ requires pairwise comparison data to train both heads, making standard SFT datasets unusable. Additionally, RM training can incorporate feedback from a list of rankings, e.g. $y_1 > y_2 > y_3$, by making pairs for all ranking combinations. For π^{hydra}, only pairs containing the sample with the best ranking should be considered to avoid training the SFT head on other samples.

Dynamic LoRA We introduce Dynamic LoRA as a simple and helpful technique to conserve memory usage in LoRA-PPO. Because π_θ and π_{ref} are initialized as copies of π_{SFT}, training π_θ with LoRA [14] means the only difference between them is the LoRA weights. Rather than loading π_{SFT} twice, π_{ref} can be recovered from the actor by "turning off" LoRA. Thus, we define $\pi_{\text{ref}} \leftarrow \text{LO}(\pi_\theta)$, where LO ignores any LoRA parameters. We add $r_\phi(x, y) \leftarrow \text{LO}(V(x, y))$ for the Critic/Reward pair, saving about 20% of memory while maintaining performance equivalent to LoRA-PPO.

Stage 2: Hydra-PPO Two separate sets of LoRA weights are added to the same base model π^{hydra}, one set for the actor and one set for the critic, in order to create $\pi_\theta^{\text{RL-hydra}}$. When the actor is required, only the actor LoRA weights are used, and similarly for the critic. Utilizing dynamic LoRA, we define $(\pi_{\text{ref}}^{\text{hydra}}, r_\phi^{\text{hydra}}(x, y)) \leftarrow \text{LO}(\pi_\theta^{\text{RL-hydra}})$. Only one full base model is required in memory during PPO, leading to similar overall memory usage to LoRA finetuning given the same batch size.

As an ablation study, we also include results of Joined Hydra-PPO or J-Hydra-PPO, which uses only one set of LoRA weights for both actor and critic. While this saves a small amount of memory and run-time, we find that it performs worse than Hydra-PPO. This is an interesting contrast to Hydra-SFT where joining the models does not affect performance.

Method	# of Static Models	# of LoRA Weight Sets
Full Fine-Tuning PPO	4	0 (fully finetuned)
LoRA-PPO	4	2
Dynamic LoRA-PPO	2	2
Joined Hydra-PPO	1	1
Hydra-PPO	1	2

Table 2: Summary of all PPO methods and number of models.

4 Experiments

Results are presented across four datasets using Llama 7b [5] or OPT 1.3b [21]. We employ GPT-4 to evaluate model performance in general [22, 8, 23, 24], and for the summarization task, we use also ROUGE scores[25] .

In the empirical study, we evaluate five approaches: SFT, LoRA-PPO, Hydra-SFT, J-Hydra-PPO, and Hydra-PPO. Specifically, LoRA-PPO is initialized with the SFT model, while both J-Hydra-PPO and Hydra-PPO are initialized with the Hydra-SFT model. All experiment hyperparameters are listed in Appendix B. Perplexity and RM accuracy before PPO is listed in Appendix B. Our code is forked from DeepSpeed-Chat [26, 27].

The performance of PPO can be highly inconsistent due to its unstable nature and varying implementations [28, 13, 29, 11]. PPO can even reduce performance by exploiting flaws in the reward model, which we observe in the StackExchange dataset.

Results Overview Tables 3 and 9 show the expected win-rates of each method against all other methods, as evaluated by GPT-4. The findings indicate that PPO outperforms SFT on average and Hydra-PPO similarly improves Hydra-SFT. The specific win-rates per dataset are provided in detail. The performance of SFT and Hydra-SFT are comparable, suggesting that combining the RM and SFT objectives within a single model does not consistently lead to improvements or hinder the generation performance across different tasks.

Both Hydra-PPO and LoRA-PPO improve over their respective base models, however, Hydra-PPO achieves better alignment than LoRA-PPO for Llama 7b. This may be explained by the better Reward model from Hydra-SFT which enables overall better PPO performance. The detailed accuracy of the RM models in SFT and Hydra-SFT is shown in Appendix F. Overall, the study indicates that PPO improves model alignment and there is potential for further enhancing PPO performance by improving the RM.

For Learning to Summarize, we additionally evaluate their performance using ROUGE scores in Table 4, and these results consistently align with the findings presented in Table 3. An interesting observation is that the SFT-based approach typically yields better precision performance, whereas PPO-based methods substantially enhance recall. This trend could potentially be attributed to the encouragement of longer text generation during the PPO stage.

Joined-Hydra-PPO Underperformance J-Hydra-PPO, which uses only one set of LoRA weights for actor and critic, performs significantly worse than two separate sets (Hydra-PPO). We speculate this is due to combining actor and critic model amplified the unstable nature of PPO [28, 13, 29, 11]. Since J-Hydra-PPO is more memory and computation efficient than Hydra-PPO, we hope future work may improve its performance.

Method	GPT-4-LLM	Open-Source Assistant	Learning to Summarize	StackExchange	**Average**
SFT	48.18	48.35	45.95	51.73	48.55
LoRA-PPO	48.8	49.03	55.48	49.4	**50.68**
Hydra-SFT	48.48	49.65	42.63	53.23	48.50
J-Hydra-PPO	50.43	**52.05**	43.13	40.38	46.50
Hydra-PPO	**54.13**	51	**61.58**	**55.38**	**55.52**

Table 3: Llama 7b expected aggregate win-rate per method. We measure total wins and ties for each method against all other methods, then use this to calculate expected win-rate.

Model	ROUGE-1			ROUGE-L		
	Precision	Recall	F-Measure	Precision	Recall	F-Measure
SFT	90.69	13.12	21.69	75.56	11.35	18.59
LoRA-PPO	88.93	14.70	23.95	71.46	12.25	19.77
Hydra-SFT	87.86	13.27	21.42	72.92	11.42	18.27
J-Hydra-PPO	84.13	16.93	25.00	70.82	14.92	21.81
Hydra-PPO	88.91	19.21	**29.31**	72.45	16.43	**24.73**

Table 4: Llama 7b ROUGE-1 and ROUGE-L scores for all models on the Learning to Summarize dataset.

Throughput Comparison Figure 2 shows there is a roughly linear relationship between throughput and sequence length in log-space for all methods. Latency is measured as a sum of inference latency and parameter update latency per sample during PPO. As we can see from the figure, Hydra-PPO saves exponentially more time as sequence length increases. We increase batch size to max out memory usage for all methods, but use gradient accumulation to ensure the effective total batch size is the same. Hydra-PPO and J-Hydra-PPO converge at sequence length 1024 as the inference increase overtakes update latency. Table 1 shows a detailed comparison for a specific experiment.

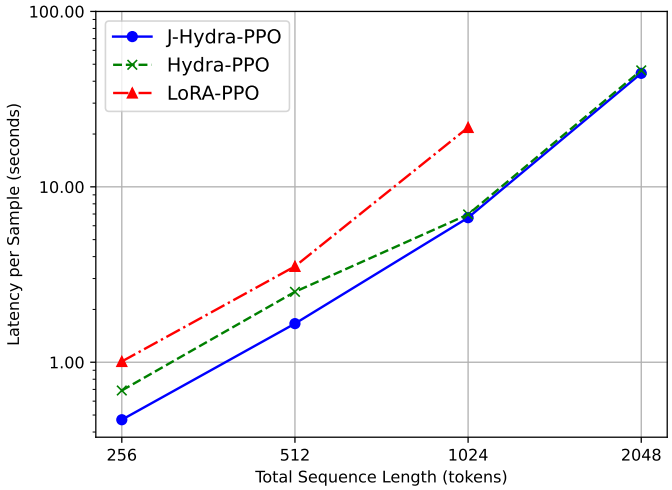

Figure 2: Latency (seconds) per Sample per PPO method as sequence length increases. Both axes use log scaling. LoRA-PPO is unable to fit in memory for our setup for context length 2048. See Appendix B for details.

4.1 GPT-4-LLM

GPT-4-LLM [22] consists of instruction-following prompts with responses sampled from multiple foundation models, including GPT-4, GPT-3.5, and OPT-IML. The responses of each model are ranked by GPT-4. We pair only the highest-scoring response with each other response. To our knowledge, we are the first to attempt full RLHF (including PPO) on this dataset. Overall, we observe the most consistent and well-behaved training runs with GPT-4-LLM.

-	Hydra-FFT	LoRA-PPO	J-Hydra-PPO	Hydra-PPO
SFT	40.6 / **43.8**	**43.6** / 43.4	43.2 / **46.8**	39.0 / **47.0**
Hydra-FFT	-	43.4 / **45.2**	40.8 / **44.0**	38.8 / **49.2**
LoRA-PPO	-	-	40.8 / **44.4**	40.0 / **47.6**
J-Hydra-PPO	-	-	-	38.6 / **45.6**

Table 5: Llama 7b GPT-4-LLM win-rates as judged by GPT-4. Results in each cell are presented as "Row Win % / Column Win %" with the remainder being ties.

4.2 Open-Source Assistant Datasets

We perform RLHF on the default data for DeepSpeed-Chat [26, 27]. At the time of writing, these datasets include "Dahoas/rm-static", "Dahoas/full-hh-rlhf", "Dahoas/synthetic-instruct-gptj-pairwise" and "yitingxie/rlhf-reward-datasets", all hosted on HuggingFace. We call the combination "Open-Source Assistant Datasets". These are various open-source ChatBot or Assistant style datasets, with one including Helpful & Harmless [9]. We train on them without modification.

-	Hydra-FFT	LoRA-PPO	J-Hydra-PPO	Hydra-PPO
SFT	42.2/ **44.4**	41.7 / **42.2**	40.6 / **45.2**	39.4 / **45.4**
Hydra-FFT	-	**45.4** / 38.4	37.8 / **45.0**	38.6 / **43.6**
LoRA-PPO	-	-	43.0 / **44.4**	42.6 / **42.8**
J-Hydra-PPO	-	-	-	**45.6** / 42.4

Table 6: Llama 7b Open-Source Assistant Datasets win-rates as judged by GPT-4. Results in each cell are presented as "Row Win % / Column Win %" with the remainder being ties.

4.3 Learning to Summarize

The Reddit TL;DR dataset [30] has been previously used in multiple RLHF works [31, 12]. We use the dataset as modified by [31], where each prompt only contains one preference completion pair.

-	Hydra-FFT	LoRA-PPO	J-Hydra-PPO	Hydra-PPO
SFT	**41.0** / 38.4	31.6 / **44.8**	**41.0** / 32.8	31.4 / **47.2**
Hydra-FFT	-	33.4 / **46.4**	**37.8** / 36.4	33.8 / **42.4**
LoRA-PPO	-	-	**51.8** / 30.0	**42.6** / 36.8
J-Hydra-PPO	-	-	-	23.7 / **52.6**

Table 7: Llama 7b Learning to Summarize win-rates as judged by GPT-4. Results in each cell are presented as "Row Win % / Column Win %" with the remainder being ties.

4.4 StackExchange

The StackExchange [32] dataset has previously been used to train StackLlama via RLHF [33]. Each data sample consists of one question with multiple completions ranked by votes from users. We re-create this experiment with 150k samples from StackExchange, with a change in that we pair only the best answer with up to 3 other answers. This is done to avoid over-training on the best sample in Hydra-SFT, but in addition, we find that the most up-voted answers are on average longer than the other answers, leading to trivial reward models.

StackExchange is the most difficult dataset we test, containing extremely diverse and specific questions. During PPO, models often learn to repeat their answers. Despite multiple attempts, both the PPO and J-Hydra-PPO models encounter this issue while Hydra-PPO does not.

-	Hydra-FFT	LoRA-PPO	J-Hydra-PPO	Hydra-PPO
SFT	41.2 / **42.4**	**46.4** / 42.0	**51.8** / 35.0	42.4 / **48.6**
Hydra-FFT	-	**46.4** / 43.2	**54.2** / 32.4	45.2 / **45.6**
LoRA-PPO	-	-	**52.6** / 34.8	36.8 / **51.8**
J-Hydra-PPO	-	-	-	35.2 / **56.6**

Table 8: Llama 7b StackExchange win-rates as judged by GPT-4. Results in each cell are presented as "Row Win % / Column Win %" with the remainder being ties.

4.5 Changing Model Size and Family

We extend our experimentation to explore the SFT and PPO approaches using the OPT-1.3b model. For this model, we find that Hydra-SFT performs worse than the SFT model. Additionally, we find LoRA-PPO has better overall alignment than Hydra-PPO for OPT-1.3b. We speculate this difference to be due to the capacity of the model. For the smaller 1.3b model, combining language and reward models may be more difficult. Overall, we observe the same trend in increased performance after PPO and Hydra-PPO over their respective base models.

Method	GPT-4-LLM	Open-Source Assistant	**Average**
SFT	45.65	52.5	49.08
LoRA-PPO	**59.5**	53.7	**56.6**
Hydra-SFT	44.4	42.58	43.49
J-Hydra-PPO	48.2	46.78	47.49
Hydra-PPO	50.2	**54.45**	**52.33**

Table 9: OPT 1.3b expected aggregate win-rate per method. We measure total wins and ties for each method against all other methods, then use this to calculate expected win-rate.

-	Hydra-FFT	LoRA-PPO	J-Hydra-PPO	Hydra-PPO
SFT	**52.2** / 36.6	41.2 / **44.0**	**52.6** / 39.4	43.2 / **49.2**
Hydra-FFT	-	35.8 / **53.4**	42.4 / **49.8**	37.8 / **56.6**
LoRA-PPO	-	-	**50.8** / 41.2	46.2 / **46.6**
J-Hydra-PPO	-	-	-	39.2 / **49.6**

Table 10: OPT 1.3b GPT-4-LLM win-rates as judged by GPT-4. Results in each cell are presented as "Row Win % / Column Win %" with the remainder being ties.

-	Hydra-FFT	LoRA-PPO	J-Hydra-PPO	Hydra-PPO
SFT	**45.0** / 44.0	31.8 / **54.8**	41.2 / **47.4**	39.8 / **50.0**
Hydra-FFT	-	34.6 / **53.0**	32.0 / **54.6**	42.4 / **44.2**
LoRA-PPO	-	-	**53.0** / 35.8	**52.0** / 33.6
J-Hydra-PPO	-	-	-	39.0 / **47.0**

Table 11: OPT 1.3b Open-Source Assistant Datasets Preference as judged by GPT-4. Results in each cell are presented as "Row Win % / Column Win %" with the remainder being ties.

5 Related Works

Aligning to Human Preference Foundation models have begun to emerge as all-purpose language models [6] which may be used without any domain adaptation [34, 1, 35]. While these models clearly contain a large amount of knowledge and ability, they may contain unintended bias or respond in unintended ways to input questions from a user. Model alignment is the problem of slightly modifying these models to interact with humans in a specific manner.

Human preference is difficult to quantify (and often inconsistent [13, 10]), making model alignment an open research area [36]. By assuming that classification is easier than generation, it is possible to train a reward model on a dataset of human preference labels. Such a reward model may then be used to guide other models towards aligning to human preference, improving performance in a nontrivial way over Supervised fine-tuning (SFT) throughout many domains [37, 12, 9, 31, 8, 38, 39]. Recently, this concept has exploded in popularity due to the success of InstructGPT and subsequent improvements in ChatGPT and GPT-4 [10] which have delivered undeniably strong and human-like interactions in a variety of domains.

Other forms of feedback have been attempted due to the high cost of hiring humans to grade inputs. Now that massive foundation models exist, multiple works have attempted to use their feedback to train or evaluate other models [22, 8, 23, 24, 40, 41].

Alignment during Supervised Fine-Tuning (SFT) Due to the complexity and high cost of PPO, some recent works have sought to replace the training process of PPO while retaining its benefits. Notably, RAFT [42], RRHF [29], PRO [43], and DPO [13] are recent methods which combine preference data in some way with supervised fine-tuning. The former two are inspired by best-of-n sampling methods [44, 45, 46], while the latter two seek to wholly replace PPO by re-framing the supervised training objective.

Hydra-SFT shares similarities with these approaches by integrating ranked feedback into supervised fine-tuning. However, our work is orthogonal to these methods, aiming not to replace RLHF, but rather to make it more widely usable.

Dataset Formation Hydra-RLHF requires that the SFT and RM training datasets be the same. Previous works have found issues in over-fitting one of the heads when data is imbalanced [12, 31]. Our experiments use datasets with pairwise comparisons for each sample so we find this over-fitting is not an issue, however, Hydra-RLHF could be extended to handle exceptions when data is limited.

Reward Model Size In RLHF, the reward model can be smaller than the language model. We keep these models the same size to make performance comparisons fair. In applied usage, Hydra-RLHF comparatively saves less memory when standard RLHF uses a smaller reward model, however, this is also an advantage for Hydra-RLHF; it uses a larger reward model for less training cost.

6 Conclusion

We have performed a comparative study which analyzes the performance of different approaches to model alignment as graded by GPT-4. We find that LoRA-PPO improves alignment over FFT but is costly to run. We introduce Hydra-RLHF as a method to save memory during PPO while maintaining performance, which consists of two major parts: combining reference and reward models, and dynamically switching the active LoRA module during PPO. With the excess memory, Hydra-RLHF may use a higher batch size and therefore train with up to 65% faster per-sample latency. Hydra-RLHF opens up the possibility for the community to apply RLHF for a wider variety of models and applications. We also see potential for future improvements, notably, balancing the SFT and RM datasets, improving performance of J-Hydra-PPO, and improving PEFT methods for RLHF.

Acknowledgments

Thank you to Vladimir Fomenko and Jialei Chen for helpful discussions.

References

[1] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with gpt-4, 2023.

[2] Chengwei Qin, Aston Zhang, Zhuosheng Zhang, Jiaao Chen, Michihiro Yasunaga, and Diyi Yang. Is chatgpt a general-purpose natural language processing task solver?, 2023.

[3] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2023.

[4] Md Tahmid Rahman Laskar, M Saiful Bari, Mizanur Rahman, Md Amran Hossen Bhuiyan, Shafiq Joty, and Jimmy Xiangji Huang. A systematic study and comprehensive evaluation of chatgpt on benchmark datasets, 2023.

[5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

[6] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto,

Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models, 2022.

[7] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, FAccT '21, page 610–623, New York, NY, USA, 2021. Association for Computing Machinery.

[8] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional ai: Harmlessness from ai feedback, 2022.

[9] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022.

[10] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[12] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences, 2020.

[13] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.

[14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[15] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018.

[16] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning, 2016.

[17] Sebastian Ruder. An overview of multi-task learning in deep neural networks, 2017.

[18] Michael Crawshaw. Multi-task learning with deep neural networks: A survey, 2020.

[19] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, jan 2016.

[20] Yannis Flet-Berliac and Philippe Preux. Merl: Multi-head reinforcement learning, 2020.

[21] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

[22] Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with gpt-4, 2023.

[23] Sungdong Kim, Sanghwan Bae, Jamin Shin, Soyoung Kang, Donghyun Kwak, Kang Min Yoo, and Minjoon Seo. Aligning large language models through synthetic feedback, 2023.

[24] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment, 2023.

[25] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.

[26] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song, and Yuxiong He. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales, 2023.

[27] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '20, page 3505–3506, New York, NY, USA, 2020. Association for Computing Machinery.

[28] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on ppo and trpo, 2020.

[29] Zheng Yuan, Hongyi Yuan, Chuanqi Tan, Wei Wang, Songfang Huang, and Fei Huang. Rrhf: Rank responses to align language models with human feedback without tears, 2023.

[30] Michael V"olske, Martin Potthast, Shahbaz Syed, and Benno Stein. TL;DR: Mining Reddit to learn automatic summarization. In *Proceedings of the Workshop on New Frontiers in Summarization*, pages 59–63, Copenhagen, Denmark, September 2017. Association for Computational Linguistics.

[31] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback, 2022.

[32] Nathan Lambert, Lewis Tunstall, Nazneen Rajani, and Tristan Thrush. Huggingface h4 stack exchange preference dataset, 2023.

[33] Edward Beeching, Younes Belkada, Kashif Rasul, Lewis Tunstall, Leandro von Werra, Nazneen Rajani, and Nathan Lambert. Stackllama: An rl fine-tuned llama model for stack exchange question and answering, 2023.

[34] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models, 2022.

[35] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[36] Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: a research direction, 2018.

[37] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences, 2023.

[38] Amelia Glaese, Nat McAleese, Maja Trębacz, John Aslanides, Vlad Firoiu, Timo Ewalds, Maribeth Rauh, Laura Weidinger, Martin Chadwick, Phoebe Thacker, Lucy Campbell-Gillingham, Jonathan Uesato, Po-Sen Huang, Ramona Comanescu, Fan Yang, Abigail See, Sumanth Dathathri, Rory Greig, Charlie Chen, Doug Fritz, Jaume Sanchez Elias, Richard Green, Soňa Mokrá, Nicholas Fernando, Boxi Wu, Rachel Foley, Susannah Young, Iason Gabriel, William Isaac, John Mellor, Demis Hassabis, Koray Kavukcuoglu, Lisa Anne Hendricks, and Geoffrey Irving. Improving alignment of dialogue agents via targeted human judgements, 2022.

[39] Jeff Wu, Long Ouyang, Daniel M. Ziegler, Nisan Stiennon, Ryan Lowe, Jan Leike, and Paul Christiano. Recursively summarizing books with human feedback, 2021.

[40] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[41] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[42] Hanze Dong, Wei Xiong, Deepanshu Goyal, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. Raft: Reward ranked finetuning for generative foundation model alignment, 2023.

[43] Feifan Song, Bowen Yu, Minghao Li, Haiyang Yu, Fei Huang, Yongbin Li, and Houfeng Wang. Preference ranking optimization for human alignment, 2023.

[44] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

[45] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. A general language assistant as a laboratory for alignment, 2021.

[46] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. Webgpt: Browser-assisted question-answering with human feedback, 2022.

[47] DeepSpeed. Deepspeed-chat. `https://github.com/microsoft/DeepSpeed/tree/master/blogs/deepspeed-chat`, 2023.

[48] Xianghui Sun, Yunjie Ji, Baochang Ma, and Xiangang Li. A comparative study between full-parameter and lora-based fine-tuning on chinese instruction data for instruction following large language model, 2023.

[49] George Pu, Anirudh Jain, Jihan Yin, and Russell Kaplan. Empirical analysis of the strengths and weaknesses of peft techniques for llms, 2023.

[50] Vladislav Lialin, Namrata Shivagunde, Sherin Muckatira, and Anna Rumshisky. Stack more layers differently: High-rank training through low-rank updates, 2023.

[51] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, and Sayak Paul. Peft: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`, 2022.

A Algorithm Pseudocode

We present pseudocode for J-Hydra-RLHF and Hydra-RLHF to aid in their implementation.

Algorithm 1 J-Hydra-PPO

for iteration=1,2,... **do**
  for actor=1,2,...N **do**
    Generate sequence using policy $\pi^{\text{hydra}}_{\theta_{\text{old}}}$ for T sequence length steps, retaining critic head values for the last step
    Turn LoRA off for $\pi^{\text{hydra}}_{\theta_{\text{old}}}$ and use it to compute reward and reference log probabilites
    Use reward and reference log probability to calculate $\hat{A}$
  end for
  Optimize surrogate $\mathcal{L}$ wrt $\pi^{\text{hydra}}_{\theta}$, with K epochs and minibatch size $M \leq NT$
  $\pi^{\text{hydra}}_{\theta_{\text{old}}} \leftarrow \pi^{\text{hydra}}_{\theta}$
end for

Algorithm 2 Hydra-PPO

for iteration=1,2,... **do**
  for actor=1,2,...N **do**
    Generate sequence using policy $\pi^{\text{hydra}}_{\theta_{\text{old}}}$ for T sequence length steps
    Turn on the critic LoRA for $\pi^{\text{hydra}}_{\theta_{\text{old}}}$ and use it to calculate critic values on the generated sequence
    Turn LoRA off for $\pi^{\text{hydra}}_{\theta_{\text{old}}}$ and use it to compute reward and reference log probabilites
    Use reward and reference log probability to calculate $\hat{A}$
  end for
  Optimize surrogate $\mathcal{L}$ wrt $\pi^{\text{hydra}}_{\theta}$, with K epochs and minibatch size $M \leq NT$
  $\pi^{\text{hydra}}_{\theta_{\text{old}}} \leftarrow \pi^{\text{hydra}}_{\theta}$
end for

B Experiment Details

Setup All hyper-parameters used are listed in Tables 12 and 13. All experiments ran on a 8xA100 80GB node. and batch size is measured per-GPU, making total batch size = 8 * batch size * gradient accumulation steps. All experiments use DeepSpeed ZeRO-1 [27, 47]. All experiments use a cosine decaying learning rate scheduler.

We fix a common set of parameters. For all experiments, for SFT, RM, and Hydra-SFT, weight decay is set to 0.1. Reward models are normalized between (-1, 1). When using LoRA, SFT models use a LoRA rank of 128. Across all SFT, RM, and Hydra-SFT experiments, we began by sweeping learning rates {5e-5, 5e-6, and 5e-7} for 4 epochs, then adjusted parameters accordingly. We use the model with the best validation performance across epochs.

For all experiments, PPO, J-Hydra-PPO, and Hydra-PPO share the following: KL-Divergence β = 0.02, GAE γ = 1.0, GAE λ = 0.95, warmup steps = 100, LoRA rank for actor and critic = 128, and all are run for 1 epoch. We track the mean of the reward from the last 20 steps, and use the model with the best reward throughout training.

Run-Time Measurement All run-time numbers were obtained from training Llama 7b on StackExchange. For Table 1, we use a total sequence length of 800 tokens. Total GPU memory exceeds the sum of Model and Activation due to other stored tensors used in training, while Total Latency per Sample is a direct sum of the two stages of PPO. Memory usage was measured with PyTorch. Because Full PPO overflows our setup, we repeat the experiment using OPT 1.3b, and use the ratio between LoRA-PPO on Llama 7b to scale the numbers up.

PPO Hyperparameter Stability For all PPO runs, we ensure reward increases during training and take the point of the highest reward without extreme and obvious divergence. In general, we find J-Hydra-PPO to be highly unstable, taking multiple attempts to find solid hyperparameters. Hydra-PPO is generally as stable as LoRA-PPO.

Hyperparameter	GPT-4-LLM	Learning to Sum.	Open-Source Assistant	StackExchange
Prompt Sequence Length	256	512	256	400
Completion Sequence Length	256	256	256	400
SFT				
Learning Rate	1e-5	1e-4	1e-3	1e-4
Batch Size	4	3	4	1
Gradient Accum. Steps	1	3	1	6
Epochs	3	7	7	4
RM				
Learning Rate	5e-5	5e-5	5e-4	1e-6
Batch Size	4	1	3	1
Gradient Accum. Steps	1	4	1	12
Epochs	3	3	6	3
Hydra-SFT				
Learning Rate	3e-6	5e-7	1e-5	5e-7
Reward Head Multiplier	0.1	0.1	0.1	0.07
Batch Size	4	1	4	1
Gradient Accum. Steps	1	10	1	6
Epochs	3	4	4	6
LoRA-PPO				
Actor Learning Rate	5e-4	8e-4	5e-4	5e-4
Critic Learning Rate	5e-5	1e-4	5e-5	5e-5
Generation Batch Size	6	2	6	1
Mini-Batch Size	8	4	8	3
Gradient Accum. Steps	5	30	3	25
J-Hydra-PPO				
Actor Learning Rate	5e-3	2.5e-4	1e-4	6e-4
Critic Loss Multiplier	0.1	3	0.1	3
Generation Batch Size	6	2	6	1
Mini-Batch Size	8	4	8	3
Gradient Accum. Steps	5	50	3	25
Hydra-PPO				
Actor Learning Rate	5e-3	5e-4	5e-5	7e-4
Critic Learning Rate	5e-5	5e-5	5e-6	8e-4
Generation Batch Size	6	2	6	1
Mini-Batch Size	8	4	8	3
Gradient Accum. Steps	5	30	3	25

Table 12: Hyperparameters for Llama 7b on all datasets.

Hyperparameter	GPT-4-LLM	Open-Source Assistant
Prompt Sequence Length	256	256
Completion Sequence Length	256	256
SFT		
Learning Rate	5e-5	1e-6
Batch Size	4	4
Gradient Accum. Steps	1	1
Epochs	3	4
RM		
Learning Rate	5e-5	1e-5
Batch Size	4	4
Gradient Accum. Steps	4	4
Epochs	3	3
Hydra-SFT		
Learning Rate	3e-6	5e-5
Reward Head Multiplier	.1	.1
Batch Size	4	4
Gradient Accum. Steps	1	1
Epochs	3	4
LoRA-PPO		
Actor Learning Rate	2.5e-4	5e-4
Critic Learning Rate	5e-5	5e-5
Generation Batch Size	6	6
Mini-Batch Size	8	8
Gradient Accum. Steps	5	3
J-Hydra-PPO		
Actor Learning Rate	5e-4	5e-4
Critic Loss Multiplier	0.1	0.1
Generation Batch Size	6	6
Mini-Batch Size	8	8
Gradient Accum. Steps	5	3
Hydra-PPO		
Actor Learning Rate	5e-4	5e-4
Critic Learning Rate	5e-5	1e-5
Generation Batch Size	6	6
Mini-Batch Size	8	8
Gradient Accum. Steps	5	3

Table 13: Hyperparameters for OPT 1.3b on all datasets.

C GPT-4 Grading

Significant consideration was given to the prompt for grading model responses. We experimented with one prompt which contained two answers to decide from (e.g. "out of answer one or two, which is better"). However, we found that simply switching the order of the answers would switch the preferred answer by a large margin.

Instead, we find the most consistent evaluation method to be grading each output on a 0-9 scale with step-by-step reasoning, and then comparing these grades side-by-side. Each output is graded 3 times at temperature 1 and the results are averaged for a final score.

When grading, we perform cleanup on answers from all models: removing EOT tokens, stopping generation upon detection of the model speaking for the "other side" (i.e. begins writing a new instruction and response for Open-Datasets questions), and stopping generation upon detection of

a 5-gram phrase sequence already found in the sequence. We evaluate using 500 samples from the validation set of each dataset.

To grade outputs on the GPT-4-LLM, Open-Datasets, and StackExchange datasets, we use the following prompt:

System Prompt:
You are an unbiased and harsh critic who judges the quality of answers from an assistant.

User Message:
Given a question from a user, rate the given answer from an assistant with an integer score out of 9. Your rating can be any number from 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9. First, give an extremely short, 1-sentence evaluation of the answer. Do not give bullet points. Do not write more than one sentence. Only write one total sentence evaluating the answer. A better answer is one that is more helpful, answers the question more precisely, is more concise, is grammatically correct, is kinder and more friendly, and does not contain any rude or confrontational speech. Give your answer in exactly this format and then end the answer:

Let us think step-by-step.
[1 sentence evaluating the quality of the answer]
The integer score out of 9 for this answer is: X

Here is the original conversation and then question from the user:

Insert conversation and question

Here is the answer from the assistant:

Insert answer to judge

To grade outputs from Learning to Summarize, we use the following prompt:

System Prompt:
You are an unbiased and harsh critic who judges the quality of summaries of a piece of text.

User Message:
Given a summary of a piece of text, rate the quality of the summary with an integer score out of 9. Your rating can be any number from 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9. First, give an extremely short, 1-sentence evaluation of the summary. Do not give bullet points. Do not write more than one sentence. Only write one total sentence evaluating the summary. A better summary is one that is more concise, captures more of the sentiment of the input text, does not contain any false information about the input, and copies minimal text directly from the input. Give your answer in exactly this format and then end the answer:

Let us think step-by-step.
[1 sentence evaluating the quality of the summary]
The integer score out of 9 for this answer is: X

Here is the original text:

Insert original text

Here is the summary of the text:

Insert summary to judge

D Reward Model Grading

We use GPT-4 as our primary evaluator, however, it is possible to use a reward model as the evaluator. Table 14 shows the win-rates of FFT-SFT vs. LoRA-PPO using the separated RM, and Hydra-SFT vs. J-Hydra-PPO and Hydra-SFT vs. Hydra-PPO using the Hydra-RM. This table shows the success of each PPO method in optimizing their actors towards their reward model. Overwhelmingly, the RMs rate each PPO model as strongly superior to its respective input model.

Dataset	LoRA-PPO	J-Hydra-PPO	Hydra-PPO
OPT 1.3b			
GPT-4-LLM	19.4 / **70.2**	20.4 / **79.6**	13.4 / **86.6**
Open-Source Assistant	20.4 / **78.6**	25.8 / **41.2**	26.8 / **40.8**
Llama 7b			
GPT-4-LLM	13.6 / **82.6**	36.4 / **62.2**	27.8 / **69.2**
Open-Source Assistant	12.4 / **14.8**	15.0 / **40.2**	21.6 / **64.4**
Learning to Summarize	34.8 / **53.8**	**44.4** / 31.0	**54.6** / 40.8
StackExchange	10.2 / **89.8**	**54.0** / 45.6	11.0 / **89.0**

Table 14: Win Rates of each PPO method versus its input model (FFT-SFT / LoRA-PPO, Hydra-SFT / J-Hydra-PPO, and Hydra-SFT / Hydra-PPO). Results in each cell are presented as "Base Wins / PPO wins" with the remainder being ties.

E LoRA-SFT vs. LoRA-PPO

For our experiments, we use FFT-SFT for both standard and hydra models. Table 15 shows results of using LoRA-SFT against LoRA-PPO, as graded by GPT-4. Here, LoRA-PPO uses FFT as the input model. We find LoRA SFT under-performs compared to FFT. While efficient, LoRA does not always match FFT-SFT performance [14, 48, 49, 50]. More investigation into PEFT [51] for alignment is required. If a method like [50] could improve LoRA-SFT for alignment, the entire RLHF process could be done with roughly the same footprint as LoRA-SFT.

Dataset	LoRA-SFT / LoRA-PPO
OPT 1.3b	
GPT-4-LLM	42.0 / **45.2**
Open-Source Assistant	29.2 / **61.0**
Llama 7b	
GPT-4-LLM	35.2 / **53.0**
Open-Source Assistant	**47.7** / 42.1
Learning to Summarize	28.2 / **50.7**
StackExchange	**45.9** / 43.1

Table 15: Win Rates of LoRA-SFT vs. LoRA-PPO (with FFT-SFT base) on all datasets using GPT-4 as a judge. Results in each cell are presented as "LoRA-SFT Wins / LoRA-PPO wins" with the remainder being ties.

F Pre-PPO Performance

We include here the results of the SFT, RM, and Hydra-SFT models on their respective datasets. Notably, the Hydra-SFT RM head usually outperforms the standard RM.

Model	Causal PPL	Reward Accuracy
GPT-4-LLM		
OPT 1.3b Separate	1.71	92.89%
OPT 1.3b Hydra	1.66	92.48%
Llama 7b Separate	1.48	93.50%
Llama 7b Hydra	1.47	95.37%
Open-Assistant		
OPT 1.3b Separate	2.00	77.79%
OPT 1.3b Hydra	1.72	80.97%
Llama 7b Separate	1.33	76.75%
Llama 7b Hydra	1.33	85.51%
Learning to Summarize		
Llama 7b Separate	2.69	69.31%
Llama 7b Hydra	2.69	69.36%
StackExchange		
Llama 7b Separate	2.08	63.90%
Llama 7b Hydra	2.10	64.20%

Table 16: Model Pre-PPO Performance results on their respective validation datasets.